

Chapter 2

Classes and Objects

An *object* is a collection of some internal state alongside some *methods* (functions that belong to that object) that can be used to query, manipulate, or use that state. An object is an instance of a *class*, which defines what the state is, what those methods are, and what other functionality is logically related to those kinds of objects. The functions that we have written so far have all been in this last category: methods that are marked `static` do not belong to a particular object. However, nonstatic methods, called *instance methods*, do belong to a particular object and have access to all of its internal state.

In the previous chapter, when converting a recursive squaring function into a loop form, we used a private helper function, while the main squaring function itself was public. Keeping the internal state and implementation details of an object private is known as *encapsulation*, one of the fundamental principles of object-oriented design. With strong encapsulation, you can guarantee that users of your class do not depend on implementation details that might change in the future. (For instance, we removed the helper function entirely upon completing the transition to loop form!) You also know that the internal state is only modified through the methods you provide, so you can enforce *invariants*: facts that are true of any valid state.

So far, we have seen public and private methods. In total, there are four levels of access that can be specified:

- `public` items are accessible by any other class.
- `private` items are accessible only within the containing class.
- `protected` items are accessible by subclasses and any other classes in the same package; we will discuss subclasses later.
- Items with no access specifier are accessible by classes in the same package.

A *package* is a collection of related classes. You can specify that your class belongs to a package using the `package` keyword, but we will primarily continue using the default package.

2.1 Input and Output

The `System` class, known in full as `java.lang.System`, is much like the program classes that we have made so far: it is not meant to be instantiated (there is no object of type `System`), but it contains a bunch of static variables and static methods to interact with the Java execution environment. One of these static variables is `out`, an object of type `PrintStream` (in full: `java.io.PrintStream`) which represents the destination for textual output of the program. (Typically, this destination would be the terminal in which the program is running, but a powerful aspect of the command-line interface is the ability to chain programs into one another, or to redirect their input or output to files instead of the terminal itself.)

Part of the internal state of a `PrintStream` is a reference to the file or device it is bound to. There are no static methods of this class, but the instance methods interact with this file or device and send appropriate commands to put text there. The primary methods that we are concerned with are as follows:

- `public void print(boolean b)`
- `public void print(char c)`
- `public void print(char[] s)`
- `public void print(double d)`
- `public void print(float f)`
- `public void print(int i)`
- `public void print(long l)`
- `public void print(Object obj)`
- `public void print(String s)`
- `public void println(boolean b)`
- `public void println(char c)`
- `public void println(char[] s)`
- `public void println(double d)`
- `public void println(float f)`
- `public void println(int i)`
- `public void println(long l)`
- `public void println(Object obj)`
- `public void println(String s)`
- `public void println()`

This is nineteen distinct methods dedicated to writing output, and it is not even an exhaustive list. The ones named `print` write out the appropriate text and leave the cursor in place immediately after that text, while the ones named `println` go one step further and move the cursor to the beginning of a new line. This is why there is a zero-argument version of `println` only; that one moves to a new line but has no other output. A zero-argument version of `print` would be useless. Many of the primitive types – `boolean`, `char`, `double`, `float`, `int`, and `long` – have their own special handling. The small integer types, `byte` and `short`, can be automatically converted to `int`, so they can still be printed. An array of characters (the `char[] s`) is treated like a `String`, while other kinds of arrays are not given nice handling. Every object in Java has a `toString` instance method, so arbitrary objects (including arrays other than `char[]`) are handled by using their `toString`

method to convert them to strings. There is special handling for null references: as a null reference does not actually refer to a valid object, calling an instance method such as `toString` is illegal, so this situation is detected and the text “null” is printed in that case. For those familiar with the Python programming language, one difference between the Python output system and the Java output system is that Java’s `print` and `println` are distinct, and another difference is that neither can take multiple arguments.

Another static variable exposed by the `System` class is `in`, an `InputStream` (in full: `java.io.InputStream`) which represents the source for input to the program. Like the `PrintStream` known as `out`, this typically refers to the terminal in which the program is run, but could be redirected to another file or the output of another program. Specifically, an `InputStream` is a source of bytes, which is not really compatible with Java’s view of strings as sequences of Unicode characters. An arbitrary byte sequence might be an invalid Unicode string, but might still be valid input to a program that wants to read, say, an image instead of a text file. A more useful interface is provided by the `java.util.Scanner` class.

A `Scanner` reads some input source and can directly provide strings or values of primitive types. It can be constructed from a `InputStream`, from a file or file path, or even from a `String`. It keeps track of how far into its input source it has read, and by default reads in chunks delimited by whitespace. To construct an instance of a class, you use the `new` keyword.

<building a scanner 25a>≡

```
java.util.Scanner in = new java.util.Scanner(System.in);
```

25a

To obviate the need to type the package name `java.util`, you can place an `import` statement at the top of your file, outside of the class definition.

<importing java.util.Scanner 25b>≡

```
import java.util.Scanner;
```

25b

Then you can just use `Scanner` instead of `java.util.Scanner`.

A `Scanner` has many helpful instance methods. There is `hasNext()` which returns a `boolean` indicating whether there is more text available to read. For each primitive type, there is a similar `hasNext...()` method that indicates whether there is more text available to read and the next chunk can be interpreted as the given type, like `hasNextInt()`, `hasNextLong()`, or `hasNextDouble()`. By using `next` instead of `hasNext`, you can get the actual value: `next()` returns a `String`, `nextInt()` return an `int`, `nextLong()` returns a `long`, `nextDouble()` returns a `double`, and so on. You can get an entire line at once with `nextLine()` (returning a `String`), and there is a corresponding `hasNextLine()` that determines whether that will succeed. If you try to use a `next...()` method when the corresponding `hasNext...()` method would return `false`, you might get a `java.util.NoSuchElementException` (if the end of the input has been reached) or a `java.util.InputMismatchException` (if the next chunk of input cannot

be interpreted as the indicated type). There is also a `close()` method that closes the `Scanner` and its underlying file. When building a `Scanner` over `System.in`, you should not use this method, as `System.in` is managed by the Java runtime environment and will be closed by Java itself at program exit. However, when constructing a `Scanner` over other files, it is convenient to close both the `Scanner` and its associated file at the same time.

The following piece of code uses some of these input and output methods to read in a series of whitespace-separated integer values and then write them out, one per line, in the reverse of the order in which they were encountered.

```
26a <IOTest.java 26a>≡
    import java.util.Scanner;
    public class IOTest {
        public static void main(String[] args) {
            go(new Scanner(System.in));
        }
        public static void go(Scanner in) {
            if (in.hasNextInt()) {
                int x = in.nextInt();
                go(in);
                System.out.println(x);
            }
        }
    }
```

If you run the `IOTest` program and type `1 2 3 4 5` done as input, then it will write `5, 4, 3, 2, and 1` as output, with one number per line.

2.2 Creating and Instantiating Classes

To illustrate the parts of a class, we'll begin by making a class representing a square. Conventionally, a class name is a noun that begins with a capital letter, so we shall call it `Square`. This class will maintain the invariant that the side length of the represented square is never negative. The overall shape of the class will look like the following.

```
26b <Square.java 26b>≡
    public class Square {
        <fields of the Square class 27a>
        <constructors of the Square class 27b>
        <methods of the Square class 28a>
    }
```

2.2.1 Fields

The variables that belong to a class or object are called *fields*. The only variable that our square needs to model its internal state is the length of its side.

<fields of the Square class 27a>≡ (26b) 27a

```
private int sideLength;
```

As this field is not `static`, each `Square` has its own individual `sideLength`. It is not marked `final`, so it can be changed after it is created. However, it is marked `private` so that it cannot be directly modified by external code. This encapsulation allows the class to maintain its invariant, that the side length is never negative.

2.2.2 Constructors

A *constructor* details how to initialize a new object. The job of a constructor is to fill in values for all of the fields, to set up the initial internal state of the object. They are defined like methods whose name is the same as the class name, except that there is no return type specified (not even `void`). A constructor does not return a value, and has direct access to the fields of the object to be initialized.

<constructors of the Square class 27b>≡ (26b) 27c ▶ 27b

```
public Square(int n) {
    if (n < 0) {
        sideLength = 0;
    } else {
        sideLength = n;
    }
}
```

All necessary information to build the object should be passed as parameters. When the constructor is finished, the object should be fully set up and ready to use; never leave an object in a partially initialize state. You can call another constructor by using `this(...)` with appropriate parameters. For instance, we might want a zero-argument constructor that creates a square of side length zero.

<constructors of the Square class 27b>+≡ (26b) ◀27b 27c

```
public Square() { this(0); }
```

2.2.3 Methods

The primary functionality of a class comes from its methods. These are the functions that belong to an object, which are used to query or modify its state or values that can be derived from this state. By convention, the first word of a method name is usually a verb, and method names are written in `camelCase`: all words together without spaces, with the first letter in lowercase, and the first letter of each

subsequent word capitalized. For instance, we might want to query the side length of our square with a `getLength()` method.

```
28a <methods of the Square class 28a>≡ (26b) 28b>
    public int getLength() { return sideLength; }
```

Like constructors, methods have direct access to the fields of the object to which they belong. You can also say `this.sideLength`, for example, which allows you to access the field even if you define a local variable with the same name.

While the square does not directly store its area, this is a value that can be computed from its internal state. We provide a method to query that as well.

```
28b <methods of the Square class 28a>+≡ (26b) <28a 28c>
    public long getArea() {
        return (long)sideLength * (long)sideLength;
    }
```

We might also wish to modify the side length. This might be through setting the length to some concrete value, or by adjusting the current length by some amount.

```
28c <methods of the Square class 28a>+≡ (26b) <28b
    public void setLength(int n) {
        if (n < 0) {
            sideLength = 0;
        } else {
            sideLength = n;
        }
    }
    public void grow(int n) {
        if (n > 0) {
            if (Integer.MAX_VALUE - sideLength > n) {
                sideLength += n;
            } else {
                sideLength = Integer.MAX_VALUE;
            }
        } else {
            if (n > -sideLength) {
                sideLength += n;
            } else {
                sideLength = 0;
            }
        }
    }
}
```

The `setLength` method here is exactly like the constructor: if the given value is positive, it is used directly, but if not, zero is filled in instead. The `grow` method does more work. First, it determines whether the amount to add is positive or negative. If positive, it has to make sure that the new size does not exceed the bounds of an `int`. If there is enough room to grow, then the given value is added directly. (The syntax `a += b` is equivalent to `a = a + b`.) Otherwise, the length is clamped at

the maximum value of an `int`, helpfully provided by `Integer.MAX_VALUE`. On the other hand, if the given value is negative, then the `grow` method has to make sure that the result is positive, that is, that `sideLength + n > 0`. (Subtracting `sideLength` from both sides gives the condition actually used in the above code; think about why the condition `sideLength > -n` could not have been used, despite appearing to be mathematically equivalent.) In this case, the given value can be added directly. Otherwise, the side length is clamped to zero. This guarantees that the size is always bounded between zero and the largest possible `int`.

All objects have a `toString()` method that creates a `String` value that textually represents the object. It is recommended, but not required, that every class override the default implementation with something that makes sense for the class.

(methods for the Square class 29a)≡

29a

```
@Override
public String toString() { return "String(" + sideLength + ")"; }
```

The `@Override` tag is optional. It protects against typos by instructing the compiler to make sure that the method is actually overriding something. If you accidentally called the method `toSting` with no `r`, then the class would not compile, because the definition is not overriding anything. Without that tag, compilation would succeed, and the default `toString` method would still be used, so you might get confused when generating strings or printing values. We will discuss other methods that are often overridden later.

2.2.4 Using Custom Classes

You can use custom classes that you have defined in exactly the same way as classes that come as part of the Java system. You need not import anything if your classes share a package and the Java system can find them. We can define a tester class for our `Square` that verifies some of its functionality. To do so, we will use a tester method that takes a test name, an actual value, and an expected value, and prints out a description of the result. It updates some class variables: `tests`, the number of tests total, and `passedTests`, the number of successful checks.

(TestSquare check method 29b)≡

(30) 29b

```
private static void check(String name, long x, long target) {
    tests++;
    String line = name + ": " + x;
    if (x == target) {
        System.out.println(" " + line);
        passedTests++;
    } else {
        line += " (expected " + target + ")";
        System.out.println("X " + line);
    }
}
```

The tester class is a program class, having a main method. It begins by creating a value of the desired type, then verifies that some queries return the expected results. After modifying the state, it executes the queries again. A good set of tests exercises all possible code paths. The limited set shown here leaves some paths unexplored. Can you see which options have not been covered?

```

30 <SquareTest.java 30>≡
    public class SquareTest {
        private static int tests;
        private static int passedTests;
        public static void main(String[] args) {
            Square square = new Square(5);
            tests = 0;
            passedTests = 0;
            check("Square(5) length", square.getLength(), 5);
            check("Square(5) area", square.getArea(), 25L);
            square.grow(3);
            check("length after grow(3)", square.getLength(), 8);
            check("  area", square.getArea(), 64L);
            square.grow(-9);
            check("length after grow(-9)", square.getLength(), 0);
            check("  area", square.getArea(), 0L);
            square.setLength(4);
            check("length after setLength(4)", square.getLength(), 4);
            check("  area", square.getArea(), 16L);
            String line = passedTests + "/" + tests + " tests passed";
            System.out.println(line);
        }
        <TestSquare check method 29b>
    }

```

This tester class uses the new `Square(5)` syntax to create a new `Square`, calling its single-parameter constructor. The `getLength()` and `getArea()` query members are used, as well as the `grow(...)` and `setLength(...)` modification members.

2.3 Summary

A class describes the internal state and available functionality of objects instantiating that class. Using encapsulation, the act of hiding implementation details (by marking some internal state and functionality as private), the class can enforce invariants, like how our `Square` ensures that its side length is always a valid nonnegative `int`. Classes are typically named in `PascalCase`, with all words joined together without spaces and the first letter of each word capitalized. Variables, fields, and methods are typically named in `camelCase`, which is much the same except that the first letter overall is lowercase. The exception is class constants declared `static`

`final`, which are typically written entirely in capital letters with words separated by underscores, to emphasize that they are constants. It is recommended that a class name be a noun and that a method name have a verb as its first word, but this convention is not universally adhered to even within the core Java built-in packages: we shall see later that a `Collection` has a `size()` method, named with a noun rather than `getSize()` that you might expect from a verb-based convention, and the `java.text.Bidi` class is not a noun (in the documentation, the phrase “a Bidi object” is used throughout, never just “a Bidi”, suggesting that “Bidi” is an adjective.)

The Java programming language uses objects and classes as the foundation upon which everything is built; essentially all functionality is tied to a class in some way. Where input and output is exposed through bare functions in Python, Java has no such concept, and instead presents writeable output through an object of type `PrintStream`, like `System.out`, and readable input through an object of type `InputStream`, like `System.in`. The `java.util.Scanner` provides a nicer interface for reading. It allows us to determine whether there is more input to read, and if so, what kind of data it is, then allows us to read it directly as an appropriate primitive type.

You define your own classes by describing their fields (variables), their constructors, and their methods (functions). Fields and methods can be static, where they belong to the class and have no associated object, or nonstatic, where they belong to a particular object and have direct access to its internal state. You construct a new class with the `new` keyword, then the class name, then the arguments to one of its exposed constructors in parentheses, like `new Square(5)`.